

Prevent breaking event deployments

Customer demonstration | Java + Kafka + AsyncAPI

A presenter guide for Swagger Contract Testing (formerly PactFlow). Allow 20-25 minutes after completing a tenant rehearsal.

Core message: the provider can pass its own tests and still break a consumer. The compatibility gate catches that before the deployment step runs.

Presentation sequence

1. Show the consumer Pact test and generated message contract.
2. Show the provider's independent AsyncAPI schema test.
3. Publish both contracts and establish a compatible baseline.
4. Rename an event field in the provider and its schema; keep the consumer unchanged.
5. Show passing provider tests, failed cross-contract comparison, and a blocked deployment gate.
6. Restore compatibility, publish a new version, and show the gate allow deployment.

What this demo actually deploys

Nothing. The gate prints a simulated deployment message. A dedicated SaaS demo environment records simulated application versions; it does not create Kafka or deploy Java services. In a real pipeline, the actual deployment command belongs behind the same gate.

What has been completed

The V4 consumer reference, standalone provider schema test, AsyncAPI definition, and local demo script are implemented. We ran the consumer and four provider tests, rehearsed local schema failure/recovery, and validated the restored AsyncAPI document with zero errors.

The conversation also covered SaaS publication and corrected a rejected AsyncAPI upload. No successful SaaS gate result is independently recorded here. SaaS outcomes on the following pages are expected results to verify in your tenant, not captured screenshots of a completed deployment.

Explain the three checks

1 / Consumer contract test

Open `consumer-java-kafka/src/test/java/io/pactflow/example/kafka/ProductsPactTestV4.java`.

The JUnit test defines a CREATED example, deserializes it, calls the real `ProductEventListener`, and verifies `repository.save` using a mocked repository. It runs without Spring startup or Kafka. Pact writes a V4 JSON contract.

```
V4Pact pact = builder.expectsToReceive("a product created event")
    .withMetadata(metadata).withContent(body).toPact();
pact.getInteractions().forEach(interaction ->
    ((V4Interaction) interaction).addReference(
        "AsyncAPI", "operationId", "sendProductEvent"));
return pact;
```

Say: "This captures an event our consumer knows how to process. The reference links the interaction to an AsyncAPI operation."

2 / Independent provider test

Open `provider-java-kafka/src/test/java/io/pactflow/example/kafka/ProductAsyncApiTest.java` and `provider-java-kafka/asynccapi.json` side by side.

The test validates actual `ProductMessageBuilder` JSON against the `ProductEvent` payload schema. It checks CREATED, UPDATED, DELETED, topic and content type. A fourth test proves that a missing ID is rejected. No consumer Pact is loaded.

3 / Cross-contract comparison in SaaS

Swagger Contract Testing compares the consumer example and metadata with the published AsyncAPI operation. The deployment gate also considers the provider's published self-verification result. Pact matchers are not used by AsyncAPI comparison; representative examples matter. [1]

The older `ProductsKafkaProducerTest.java` loads consumer Pacts and calls `verifyInteraction()`. That is consumer-driven provider verification, a separate workflow from this BDCT demonstration.

Prepare one terminal

Before the customer joins

Use Bash or zsh, start at the repository root, and retain this terminal for all steps. You need the working workshop JDK, Python 3, Node/npm for specification validation, Docker running, and a write-enabled SaaS token. Rehearse to populate dependency caches.

```
export DEMO_ROOT="$PWD"
export PACT_BROKER_BASE_URL="https://YOUR-TENANT.pactflow.io"
export PACT_BROKER_TOKEN="YOUR-WRITE-TOKEN"
export CONSUMER="pactflow-example-consumer-java-kafka-V4"
export PROVIDER="pactflow-example-provider-java-kafka"
export DEMO_RUN="demo-$(date -u +%Y%m%dT%H%M%SZ)"
export CONSUMER_VERSION="${DEMO_RUN}-consumer"
export BASELINE_VERSION="${DEMO_RUN}-baseline"
export CANDIDATE_VERSION="$BASELINE_VERSION"
export DEMO_ENV="${DEMO_RUN}-environment"
export PACT_FILE="consumer-java-kafka/build/pacts/${CONSUMER}-${PROVIDER}.json"
mkdir -p "$DEMO_ROOT/output/demo-runs/$DEMO_RUN"

docker pull pactfoundation/pact-cli:latest
pact_cli() {
  docker run --rm \
    -v "$DEMO_ROOT:/work" -w /work \
    -e PACT_BROKER_BASE_URL -e PACT_BROKER_TOKEN \
    pactfoundation/pact-cli:latest "$@"
}
pact_cli pactflow publish-provider-contract --help
```

Expected / show

The help must support --specification asyncapi. Configure credentials before sharing your screen; never put a real token in this guide or source control.

Use `pact_cli pact-broker` for publish, can-i-deploy and environment commands. Use `pact_cli pactflow` for publish-provider-contract. Omitting `pact-broker` caused the earlier "can-i-deploy: not found" error.

All commands below use these same variable names. The timestamp versions identify this manual demo; a production pipeline should identify the exact source and artifacts tested, normally using commit-based versions.

Run and show the local baseline

Step 1 / Confirm the restored state

ProductEvent.java should have private String name with no JsonProperty("productName") annotation. In asyncapi.json, the payload properties and required list should use name. AsyncAPI message and example identifiers also use the reserved key name.

```
cd "$DEMO_ROOT"
bash bdct-demo.sh baseline
npx --yes --package @asyncapi/cli asyncapi validate \
  provider-java-kafka/asyncapi.json
```

The script runs these tests in sequence, forcing fresh execution:

```
# In consumer-java-kafka:
./gradlew test --tests 'io.pactflow.example.kafka.ProductsPactTestV4' \
  --rerun-tasks --console=plain
# In provider-java-kafka, with BDCT_SCHEMA_FILE selecting asyncapi.json:
./gradlew test --tests 'io.pactflow.example.kafka.ProductAsyncApiTest' \
  --rerun-tasks --console=plain
```

Expected / show

Terminal: Message received, BUILD SUCCESSFUL, then "Consumer passed; AsyncAPI operation reference verified". The Python check in the script checks the generated V4 reference; it does not perform SaaS comparison. Provider output shows all four tests PASSED. The missing-ID test passes because rejection is expected.

Open the generated Pact at \$PACT_FILE. Show the contents, kafka_topic metadata, and comments.references.AsyncAPI.operationId = sendProductEvent.

Open provider-java-kafka/build/reports/tests/test/index.html for readable provider results. Each provider run replaces this report. Specification validation should report zero errors; an informational suggestion to use AsyncAPI 3.1 is not a failure.

Say: "Both applications pass their own checks. Next we publish their contracts so we can assess compatibility independently."

Publish the consumer contract

Step 2 / Upload the tested V4 Pact

Continue only after the local consumer test passes. The exact file path avoids publishing an older V3 Pact from the same folder.

```
pact_cli pact-broker publish "$PACT_FILE" \  
  --consumer-app-version "$CONSUMER_VERSION" \  
  --branch bdct-demo
```

Expected / show

The CLI reports successful publication and prints a link. Open that version in Swagger Contract Testing and show the consumer/provider names, version and asynchronous interaction.

```
{  
  "comments": {  
    "references": {  
      "AsyncAPI": {  
        "operationId": "sendProductEvent"  
      }  
    }  
  }  
}
```

Say: "We publish the contract generated by the test, not the Java source. The consumer version stays fixed for the rest of the demonstration."

Keep these details consistent

The provider name comes from `@PactTestFor` and the generated Pact. The provider publication must use the same name. The AsyncAPI operation key must exactly match `sendProductEvent`. The topic is `products`.

A successful upload proves that the contract was accepted. It is not yet proof of compatibility. If authentication, version selection or operation references fail, resolve them before continuing to the deployment story.

Publish the provider and test result

Step 3 / Define this helper once

It snapshots the selected schema, tests the actual Java builder against that snapshot, saves a version-specific log, and publishes the real exit status. It publishes failed results honestly and returns the test failure to the terminal. [2]

```
publish_candidate() {
  local rel="output/demo-runs/$DEMO_RUN/$CANDIDATE_VERSION"
  local dir="$DEMO_ROOT/$rel"
  local result
  mkdir -p "$dir"
  cp "$DEMO_ROOT/provider-java-kafka/asyncapi.json" "$dir/asyncapi.json"
  if (cd "$DEMO_ROOT/provider-java-kafka" && \
    BDCT_SCHEMA_FILE="$dir/asyncapi.json" \
    ./gradlew test \
      --tests 'io.pactflow.example.kafka.ProductAsyncApiTest' \
      --rerun-tasks --console=plain) > "$dir/provider-test.log" 2>&1; then
    result=0
  else
    result=$?
  fi
  cat "$dir/provider-test.log"
  pact_cli pactflow publish-provider-contract "$rel/asyncapi.json" \
    --provider "$PROVIDER" \
    --provider-app-version "$CANDIDATE_VERSION" \
    --branch bdct-demo \
    --specification asyncapi \
    --content-type application/json \
    --verification-exit-code "$result" \
    --verification-results "$rel/provider-test.log" \
    --verification-results-content-type text/plain \
    --verifier "JUnit5-networknt-json-schema" || return $?
  return "$result"
}
publish_candidate
```

Expected / show

Four passing tests and a successful baseline provider upload. Do not edit code during the helper's run. Show the provider's AsyncAPI and passing self-verification result in SaaS. The log is retained under output/demo-runs for this version.

Say: "The provider proves its implementation conforms to its own contract. SaaS will check that contract against consumer expectations."

Establish the deployment gate

Step 4 / Confirm baseline compatibility

```
pact_cli pact-broker can-i-deploy \
  --participant "$CONSUMER" --version "$CONSUMER_VERSION" \
  --participant "$PROVIDER" --version "$BASELINE_VERSION" \
  --retry-while-unknown 12 --retry-interval 5
```

Continue only when this succeeds. Create a unique, non-production environment and record simulated baseline deployments:

```
pact_cli pact-broker create-environment \
  --name "$DEMO_ENV" --display-name "Customer BDCT demo" --no-production
pact_cli pact-broker record-deployment \
  --participant "$CONSUMER" --version "$CONSUMER_VERSION" \
  --environment "$DEMO_ENV"
pact_cli pact-broker record-deployment \
  --participant "$PROVIDER" --version "$BASELINE_VERSION" \
  --environment "$DEMO_ENV"

deploy_demo() {
  if pact_cli pact-broker can-i-deploy \
    --participant "$PROVIDER" --version "$CANDIDATE_VERSION" \
    --to-environment "$DEMO_ENV" \
    --retry-while-unknown 12 --retry-interval 5; then
    echo "DEPLOYMENT ALLOWED - simulated deployment step"
  else
    echo "DEPLOYMENT BLOCKED - deployment step was not executed"
    return 1
  fi
}
deploy_demo
```

Expected / show

Baseline: DEPLOYMENT ALLOWED. Show that the environment contains the original consumer. These commands record SaaS state, not an actual deployment.

Say: "The gate checks the candidate provider against the consumer version already deployed here. A nonzero result stops the pipeline before its deployment action." [3]

Introduce the breaking change

Step 5 / Keep the consumer unchanged

Back up only the two provider files you will modify. Run this once, before editing.

```
export DEMO_BACKUP="$DEMO_ROOT/output/demo-runs/$DEMO_RUN/backup"
mkdir -p "$DEMO_BACKUP"
cp "$DEMO_ROOT/provider-java-kafka/asyncapi.json" "$DEMO_BACKUP/asyncapi.json"
export EVENT_FILE="$DEMO_ROOT/provider-java-kafka/src/main/java/"
export EVENT_FILE="${EVENT_FILE}io/pactflow/example/kafka/ProductEvent.java"
cp "$EVENT_FILE" "$DEMO_BACKUP/ProductEvent.java"
```

In ProductEvent.java, add this annotation immediately above the existing name field:

```
@com.fasterxml.jackson.annotation.JsonProperty("productName")
private String name;
```

In asyncapi.json, make exactly these payload edits:

1. Rename components.schemas.ProductEvent.properties.name to productName.
2. Replace name with productName in that schema's required array.
3. Rename the name key inside the example's payload to productName.

Do not rename components.messages.ProductEvent.name or examples[0].name. Those are AsyncAPI metadata, not event fields. Their values can remain ProductEvent and productCreated. Do not globally replace the word name.

```
cd "$DEMO_ROOT"
npx --yes --package @asyncapi/cli asyncapi validate \
  provider-java-kafka/asyncapi.json
export CANDIDATE_VERSION="${DEMO_RUN}-breaking"
publish_candidate
```

Expected / show

The document is valid and all four provider tests still pass. Java now emits productName and its schema requires productName. The consumer still expects name. Do not regenerate or republish the consumer.

Say: "This is an internally consistent provider release. Its own tests are green, but it has changed an existing integration contract."

Show deployment being blocked

Step 6 / Run the exact same gate

```
deploy_demo
```

Expected / show

The CLI should report a failed compatibility result and print:

```
DEPLOYMENT BLOCKED - deployment step was not executed
```

Follow the verification link into Swagger Contract Testing. Show the consumer version, new provider version, and the actual comparison details. The consumer example lacks the newly required `productName`. The precise UI wording may vary.

Customer evidence checklist

1. Provider self-verification is successful: its builder matches the new schema.
2. Cross-contract comparison is failed for the original consumer and breaking provider.
3. The gate returned nonzero, so the successful deployment branch did not execute.
4. The environment still records the original provider version; we never recorded a deployment of the blocked candidate.

Say: "Unit and schema tests are green. Compatibility is red. The gate prevents this otherwise valid provider release from reaching this consumer."

What not to mistake for the demonstration

A missing result, bad token or unknown operation can also cause a nonzero gate. Show the actual message incompatibility, not just a generic failure. Wait for comparison with the retry flags and inspect the linked details.

PactFlow does not independently stop an external deployment command. The pipeline must honor the gate's status. Do not use `--dry-run` or ignore the result. In production, replace the simulated echo with the real deployment step and record-deployment only after that step succeeds.

Restore compatibility and close

Step 7 / Restore the two baseline files

```
cp "$DEMO_BACKUP/ProductEvent.java" "$EVENT_FILE"
cp "$DEMO_BACKUP/asynccapi.json" \
  "$DEMO_ROOT/provider-java-kafka/asynccapi.json"

export CANDIDATE_VERSION="${DEMO_RUN}-fixed"
publish_candidate
deploy_demo
```

Expected / show

Four provider tests pass. The new fixed contract is compatible with the unchanged consumer. The gate prints DEPLOYMENT ALLOWED. Use a new version rather than overwriting the failed one; the baseline, breaking and fixed versions remain visible in SaaS.

If you want the demo environment to show the simulated rollout, run this only after the fixed gate succeeds:

```
pact_cli pact-broker record-deployment \
  --pacticipant "$PROVIDER" --version "$CANDIDATE_VERSION" \
  --environment "$DEMO_ENV"
```

Closing talk track

"We independently tested both applications, published their contracts, and caught a breaking event change before the deployment step. Restoring compatibility made the same gate pass. No end-to-end Kafka environment was needed for these contract checks."

What this does not cover

Broker connectivity, delivery guarantees, partitions, retries, ordering and schema-registry behaviour need separate integration tests. The schema tests exercise selected event examples, not all possible provider behaviour. The consumer CREATED example does not prove every consumer event path.

In a real CI/CD pipeline, run tests and publish on each build, identify versions by tested source, check the target environment, deploy only after success, then record the successful deployment. [3]

Troubleshooting and rehearsal notes

Optional local failure example

```
bash bdct-demo.sh broken-provider
bash bdct-demo.sh provider
```

Use this only after recovery to the original name baseline. The first command creates build/bdct/asynccapi-breaking.json, requires productName, and deliberately fails three provider cases. It changes no source. The second restores a passing check by selecting the original schema.

This is provider implementation drift, not SaaS cross-contract incompatibility. Show the schema diff and provider HTML report before running recovery, because that report is replaced by the next test run.

Errors encountered and corrected

- can-i-deploy: not found: use pact_cli pact-broker can-i-deploy. Consumer publishing also needs pact-broker.
- HTTP 400 invalid AsyncAPI: only rename payload fields. Message name and example name are reserved metadata keys. Do not add \$ref merely because a combined schema error mentions it.
- Provider tests pass but AsyncAPI upload fails: payload validation is not full-document validation. Run the AsyncAPI CLI before publishing.
- named or inconsistent required fields: keep Java JSON output, schema properties and required list aligned. The restored baseline uses name everywhere in the payload.
- Variables or shell functions missing: run preparation again in one terminal. Do not source bdct-demo.sh; execute it with bash.
- Missing/unknown verification: check exact application names, operation reference, uploaded versions and tenant AsyncAPI support.

Pre-call checklist

Keep code and browser side by side. Prepare a write token off-screen. Verify the baseline in SaaS. Keep the consumer version fixed. Back up the two provider files. Rehearse the blocked and recovered candidate versions. Keep the target environment dedicated to the demo.

Reference documentation

[1] AsyncAPI comparison rules: payload, metadata and operation references.

<https://support.smartbear.com/swagger/contract-testing/docs/en/user-guide/contract-testing/bi-directional-contract-testing/supported-contracts/asynccapi/features-testing-asynccapi.html>

[2] Contract publishing: AsyncAPI and provider test results.

<https://support.smartbear.com/swagger/contract-testing/docs/en/user-guide/contract-testing/bi-directional-contract-testing/publishing-contracts.html>

[3] Deployment gating and recording deployments.

https://docs.pact.io/pact_broker/can_i_deploy